

Ch 1 Introduction

- **DB:** database a collection of interrelated data
- **DBMS:** DataBase Management System
- **Applications** Set of programs to access the data
- **Levels of Abstraction 数据抽象三个层次**

Physical level——最低层次，描述数据实际是怎样存储的

Logical level——描述存储什么数据以及数据间的关系

View level——最高层次

- **Data Independence 数据独立性**

Physical Data Independence 物理数据独立性：应用程序不依赖于物理模式。

the ability to modify the physical schema without changing the logical schema or the views

Logical Data Independence 逻辑数据独立性：the ability to modify the logical schema without changing the logical views

- **Instances and Schemas**

Schemas 模式——数据库的总体设计

Physical schema: database design at the physical level;

Logical schema: database design at the logical level;

Instances 实例——特定时刻存储在数据库中的信息的集合

- **Database Languages**

数据库系统提供**数据定义语言 (data-definition language)**来定义数据库模式，以及**数据操纵语言 (data-manipulation language)**来表达数据库的查询和更新。而实际上，数据定义和数据操纵语言并不是

数据操纵语言 DML, Data Manipulation Language

ch 2 Relational Model

1 Relational Data Structure

- 关系模型的数据结构——关系 **relation**

- **Relation Schema 关系模式**

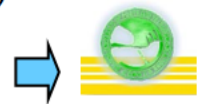
• $R = (A_1, A_2, \dots, A_n)$ is a *relation schema*

E.g. *Customer-schema* =
(*customer-name*, *customer-street*, *customer-city*)

$r(R)$ is a *relation* on the *relation schema* R

E.g. *customer* (*Customer-schema*)

$R(A_1, A_2, \dots, A_n) / R(U)$



- **Relation Instance 实例**

这样，在关系模型的术语中，**关系 (relation)** 用来指代表，而**元组 (tuple)** 用来指代行。类似地，**属性 (attribute)** 指代的是表中的列。

考察图 2-1，我们可以看出 *instructor* 关系有四个属性：*ID*、*name*、*dept_name* 和 *salary*。

我们用**关系实例 (relation instance)** 这个术语来表示一个关系的特定实例，

course_id	prereq_id
BIO-301	BIO-101
BIO-396	BIO-301
CS-190	CS-101

Order of tuples is **irrelevant** 行顺序无关

列顺序无关

属性是原子的 atomic

属性名不同

不同属性可来自相同域

行不重复

- **超键 superkey**

一个或多个属性的集合，使可以在一个 **relation** 中唯一地表示一个 **tuple**

- **候选键 candidate key**

minimal superkey 最小的超键

存在多个

- **主键、关键字 primary key**

one of the candidate keys

- **Foreign Key 外键**

For R_1, R_2 ,

R_1 includes the *Primary key* X of R_2 , X is called a **Foreign Key** 外键 referencing R_2 ;

R_1 is called the **referencing relation** 参照关系 of the foreign key dependence;

R_2 is called the **referenced relation** 被参照关系 of the foreign key ;

- **Integrity Constraint**

Integrity Constraint of Primary Key 实体完整性约束

Each specified column of **primary key** is assumed to be NOT NULL.

主键不可以为空

Referential Integrity constraint 参照完整性约束

The FK-values in any tuple of relation r_1 are either NULL or must appear as the PK-values of a tuple of relation r_2 .

在关系 r_1 中的任何元组的外键值要么是NULL，要么必须作为关系 r_2 中某个元组的主键值出现。

2 Fundamental Relational-Algebra-Operations

- **Select 选择**

$\sigma_p(r)$ 中的 p 就是 selection predicate

$$\sigma_p(r) = \{t | t \in r \text{ and } p(t)\}$$

- Defined as:

$$\sigma_p(r) = \{t \mid t \in r \text{ and } p(t)\}$$

Where p is a formula in propositional calculus consisting of terms connected by : $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

Each term is one of : G

$\langle \text{attribute} \rangle op \langle \text{attribute} \rangle$ or $\langle \text{constant} \rangle$

where op is one of: $=, \neq, >, \geq, <, \leq$

- Example of selection:

$$\sigma_{dept_name = \text{“Physics”}}(instructor)$$

- **Project 投影**

一元操作

$\Pi_{A1, A2, \dots, Ak}(r)$ 其中, $A1$ 是属性名, r 是 relation name

消去没有列出的列、消去重复的行

- **Union 并**

$r \cup s$

定义: $r \cup s = \{t | t \in r \text{ or } t \in s\}$

可并的条件:

1. 有相同的 attributes 的个数
2. 对应的取值 domains 必须相容

- **set difference 差**

$r - s$

属于 r , 不属于 s

- **Cartesian product 笛卡尔乘积**

$r \times s$

- **Rename 更名**

- Allows us to name, and therefore to refer to, the results of relational-algebra expressions.

- Allows us to refer to a relation by more than one name. Example:

$\rho_x(E)$: returns the expression E under the name X

If a relational-algebra expression E has arity n , then

$$\rho_x(A1, A2, \dots, An)(E)$$

returns the result of expression E under the name X , and with the attributes renamed to $A1, A2, \dots, An$.

3 Additional Relational-Algebra-Operations

- **Set intersection 交**

$$r \cap s$$

等于 $r - (r - s)$

条件和 union 相同

- (Natural) join (自然) 连接

Join 连接

▪ Notation: $r \bowtie_{A \theta B} s$

$$\begin{aligned} r \bowtie_{A \theta B} s &= \{t \mid t \in r \text{ and } t \in s \text{ and } (t[A] \theta t[B])\} \\ &= \sigma_{A \theta B} (r \times s) \end{aligned}$$

$$\theta \in \{=, \neq, \geq, \leq, >, <\}$$

自然连接

Notation: $r \bowtie s$

$$r \bowtie s = \Pi_{\text{Att}(r) \cup (\text{Att}(s) - \{B\})} \sigma_{r.B=s.B} (r \times s)$$

B are the same attributes of r and s;

- Division 除法 简略讲

$$r \nabla s$$

$$R = (A_1, \dots, A_m, B_1, \dots, B_n)$$

$$S = (B_1, \dots, B_n)$$

The result of $r \div s$ is a relation on schema

$$R - S = (A_1, \dots, A_m)$$

$$r \div s = \{t \mid t \in \Pi_{R-S}(r) \wedge \forall u \in s (tu \in r)\}$$

$$= \Pi_{R-S}(r) - \Pi_{R-S}((\Pi_{R-S}(r) \times s) - r)$$

- Assignment 赋值

The assignment operation ($\leftarrow$) provides a convenient way to express complex queries, write query as a sequential program consisting of a series of assignments followed by an expression whose value is displayed as a result of the query.

- **outer join 外连接**

添加没有连接上的数据

左外连接、右外连接，对于另外一边自己没有的属性，用 `null` 表示

- **Example**

找到最大的 XXX

自己笛卡尔自己，找到能够小于 XXX 的 tuple，投影一下，再删除这些

- **Find the largest account balance**

$$\Pi_{balance}(account) - \Pi_{account.balance}$$

$$(\sigma_{account.balance < d.balance} (account \times \rho_d(account)))$$

4 Modification of the Database

- **Deletion**

$$r \leftarrow r - E$$

where r is a relation and E is a relational algebra query

- **Insertion**

$$r \leftarrow r \cup E$$

where r is a relation and E is a relational algebra expression

- **Upadting**

$$r \leftarrow \Pi_{F1, F2, \dots, Fi}(r)$$

1 建表

1.1 格式

create table r(A₁D₁, A₂D₂, ..., A_nD_n, (integrity_constraint₁), ..., (integrity_constraint_k))

其中,

r is the name of the relation

each A_i is an attribute name in the schema of relation r

D_i is the data type of values in the **domain** of attribute A_i

1.2 Integrity Constraints 完整性约束

- **not null**
- **primary key** ($A_1, \dots, A_n$)
- Foreign Key ($A_1, \dots, A_n$) reference table r on delete restrict/cascade/set NULL
- **check** (P), where P is a predicate

```
create table instructor (  
    ID          char(5),  
    name        varchar(20) not null,  
    dept_name    varchar(20),  
    salary       numeric(8,2),  
    primary key (ID),  
    foreign key (dept_name) references  
    department);
```



2 Drop and Alter Table Constructs 删表&修改表

- **drop table**
- **Alter Table**

用于向现有关系添加属性。

alter table r add A D

3 Basic Structure__query 基础结构查询

- 基本语法:

- A typical SQL query has the form:

***select $A_1, A_2, \dots, A_n$
from $r_1, r_2, \dots, r_m$
where P***

A_i represent attributes

r_i represent relations

P is a predicate.

- * 表示所有的属性:

select * from instructor

- SELECT 子句可以包含涉及运算符 +、-、* 和 / 的算术表达式，并且可以对常量或元组的属性进行操作。

select ID, name, salary/12

from instructor

将返回一个与 instructor 关系相同的关系，除了属性 salary 的值被除以 12。

- **distinct**

删去重复的

select **distinct** dept_name from instructor

- **all**

不写默认保留

不删去重复的

select **all** dept_name from instructor

- **exist & not exist**

- **where & between and**

```
select name
from instructor
where salary between 90000 and 100000
等价于
select name
from instructor
where salary  $\geq$  90000 and salary  $\leq$  100000
```

- **The Rename Operation 重命名**

old_name as new_name (1)

- **自比较、自连接**

Find the names of all instructors who have a higher salary than some instructor in 'Comp. Sci'.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Comp. Sci.'
```

- **String Operations**

percent (%). The % character matches any substring;

underscore (_). The _ character matches any character;

Find the names of all instructors whose name includes the substring "dar".

```
select name
from instructor
where name like '%dar%'
```

下列例子：

- 'Intro%' 匹配任何以“Intro”打头的字符串。
- '% Comp%' 匹配任何包含“Comp”子串的字符串，例如 'Intro. to Computer Science' 和 'Computational Biology'。
- '___' 匹配只含三个字符的字符串。
- '___%' 匹配至少含三个字符的字符串。

- **Ordering the Display of Tuples**

默认是升序 (asc)

降序: desc

```
order by name desc
```

4 Set Operations

- **union**
- **intersect**
- **except**

5 Aggregate Functions

- **avg:** average value
- **min:** minimum value
- **max:** maximum value
- **sum:** sum of values
- **count:** number of values
- Aggregate Functions – **Group By**

Find the average salary of instructors in each department

```
select dept_name, avg (salary) as avg_salar  
from instructor  
group by dept_name;
```

先分组，然后在每个分组上进行指定的聚集计算

注意：出现在 select 但没有被 Aggregate 的属性，必须出现在 **group by** 里

- **Having clause**

只跟在 group by 后面出现

同样的，出现在 having 但没有被 Aggregate 的属性，必须出现在 **group by** 里

6 Nested Subqueries 嵌套子查询

- 无关子查询
- 相关子查询

6.1 Subqueries in the Where Clause

- 通常用于:

For set membership 集合成员资格

For set comparisons 集合的比较

For set cardinality

- **set comparisons**

“some” Clause: “至少比某一个要大”用 $> \text{some}$ 表示

● **Same query using $> \text{some}$ clause**

```
select name
from instructor
where salary  $> \text{some}$  (select salary
                        from instructor
                        where dept_name = 'Biology')
```

“all” Clause

SQL 也允许 $< \text{some}$, $\leq \text{some}$, $\geq \text{some}$, $= \text{some}$ 和 $<> \text{some}$ 的比较。作为练习, 请验证 $= \text{some}$ 等价于 **in**, 然而 $<> \text{some}$ 并不等价于 **not in**。☹

现在我们稍微修改一下我们的查询。找出满足下面条件的所有教师的姓名, 他们的工资值比 Biology 系每个教师的工资都高。结构 $> \text{all}$ 对应于词组“比所有的都大”。使用该结构, 我们写出查询如下:

```
select name
from instructor
where salary  $> \text{all}$  (select salary
                    from instructor
                    where dept_name = 'Biology');
```

类似于 **some**, SQL 也允许 $< \text{all}$, $\leq \text{all}$, $\geq \text{all}$, $= \text{all}$ 和 $<> \text{all}$ 的比较。作为练习, 请验证 $<> \text{all}$ 等价于 **not in**, 但 $= \text{all}$ 并不等价于 **in**。

exists & not exists: The exists construct returns the value **true** if the argument subquery is nonempty.

• The **exists** construct returns the value **true** if the argument subquery is nonempty.

• **exists** $r \Leftrightarrow r \neq \emptyset$

• **not exists** $r \Leftrightarrow r = \emptyset$

unique & not unique

- The **unique** construct tests whether a subquery has any duplicate tuples in its result.
- The **unique** construct evaluates to “true” if a given subquery contains no duplicates .
- Find all courses that were offered at most once in 2009

```
select T.course_id
from course as T
where unique (select R.course_id
               from section as R
               where T.course_id= R.course_id
               and R.year = 2009);
```

6.2 Subqueries in the From Clause

- Find the average instructors' salaries of those departments where the average salary is greater than \$42,000.”

```
select dept_name, avg_salary
from (select dept_name, avg (salary) as
      avg_salary
      from instructor
      group by dept_name)
where avg_salary > 42000;
```

6.3 With Clause

The with clause provides a way of defining a temporary relation whose definition is available only to the query in which the with clause occurs.

提供定义临时关系的方法，该定义只对包含 with 子句的查询有效

- Find all departments with the maximum budget

```
with max_budget (value) as  
  (select max(budget)  
   from department)  
select department.name  
from department, max_budget  
where department.budget = max_budget.value;
```

6.4 Subqueries in the Select Clause

SQL 允许子查询出现在返回单个值的表达式能够出现的任何地方，只要该子查询只返回包含单个属性的单个元组；这样的子查询称为**标量子查询 (scalar subquery)**。例如，一个子查询可以用到下面例子的 **select** 子句中，这个例子列出所有的系以及它们拥有的教师数：

```
select dept_name,  
       (select count( *)  
        from instructor  
        where department.dept_name = instructor.dept_name)  
       as num_instructors  
from department;
```

7 Deletion

- delete from

- 直接删表里所有数据
- 加上 `where` 语句，删除特定数据

8 Insertion

- Insert into Values ()

```
1 insert into course (course_id, title, dept_name, credits) values  
2 ('CS-437', 'Database Systems', 'Comp. Sci.', 4);
```

- 批量插入

Add all instructors to the student relation with tot_creds set to 0

```

1 insert into student
2 select ID, name, dept_name, 0
3 from instructor

```

9 Update

Increase salaries of instructors whose salary is over \$100,000 by 3%, and all others by a 5%
这里的顺序不能调换

```

1 update instructor set salary = salary * 1.03 where salary > 100000;
2 update instructor set salary = salary * 1.05 where salary <= 100000;

```

ch 4 Intermediate SQL

1 Join Expressions

- Natural Join operations 自然连接

连接条件用 `on` 连接

■ **select** *

from *course* **natural join** *prereq*

■ **select** *

from *course* **join** *prereq* **on** *course.course_id=prereq.course_id*

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>	<i>prereq_id</i>
BIO-301	Genetics	Biology	4	BIO-101
CS-190	Game Design	Comp. Sci.	4	CS-101

可以用等价的 `where` 语句替换 `join on`

we can use the **using** construct that allows us to specify exactly which columns should be equated

- Left/Right Outer Join

实际上有三种形式的外连接：

- 左外连接 (left outer join) 只保留出现在左外连接运算之前 (左边) 的关系中的元组。
- 右外连接 (right outer join) 只保留出现在右外连接运算之后 (右边) 的关系中的元组。
- 全外连接 (full outer join) 保留出现在两个关系中的元组。

相比而言，为了与外连接运算相区分，我们此前学习的不保留未匹配元组的连接运算被称作内连接 (inner join) 运算。

- `course inner join prereq on
course.course_id = prereq.course_id`

course_id	title	dept_name	credits	prereq_id	course_id
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190

- What is the difference between the above, and a natural join?

- `course left outer join prereq on
course.course_id = prereq.course_id`

course_id	title	dept_name	credits	prereq_id	course_id
BIO-301	Genetics	Biology	4	BIO-101	BIO-301
CS-190	Game Design	Comp. Sci.	4	CS-101	CS-190
CS-315	Robotics	Comp. Sci.	3	null	null

2 Views

```
create view v as <query expression>
```

- Update of a View

Add a new tuple to faculty view

```
insert into faculty  
values ( '30765', 'Green', 'Music' );
```

这个插入必须表示为对 *instructor* 关系的插入，因为 *instructor* 是数据库系统用于构造视图 *faculty* 的实际关系。然而，为了把一个元组插入到 *instructor* 中，我们必须给出 *salary* 的值。存在两种合理的解决方法来处理该插入：

- 拒绝插入，并向用户返回一个错误信息。
- 向 *instructor* 关系插入元组 ('30765', 'Green', 'Music', null)。

- 可更新的 view

一般说来, 如果定义视图的查询对下列条件都能满足, 我们称 SQL 视图是**可更新的 (updatable)** (即视图上可以执行插入、更新或删除):

- **from** 子句中只有一个数据库关系。
- **select** 子句中只包含关系的属性名, 不包含任何表达式、聚集或 **distinct** 声明。
- 任何没有出现在 **select** 子句中的属性可以取空值; 即这些属性上没有 **not null** 约束, 也不构成主码的一部分。
- 查询中不含有 **group by** 或 **having** 子句。

3 Transactions

事务 (transaction) 由查询和 (或) 更新语句的序列组成。SQL 标准规定当一条 SQL 语句被执行, 就隐式地开始了一个事务。下列 SQL 语句之一会结束一个事务:

- **Commit work:** 提交当前事务, 也就是将该事务所做的更新在数据库中持久保存。在事务被提交后, 一个新的事务自动开始。
- **Rollback work:** 回滚当前事务, 即撤销该事务中所有 SQL 语句对数据库的更新。这样, 数据库就恢复到执行该事务第一条语句之前的状态。

4 Integrity Constraints

- **not null**
- **primary key**
- **unique**
- **check (P)**, where P is a predicate

关系中的每个元组都必须满足谓词 P

- **foreign key**

on delete restrict/cascade/set NULL

cascade: 在父表上 update / delete 记录时, 同步 update / delete 掉子表的匹配记录

Restrict: 如果子表中有匹配的记录, 则不允许对父表对应候选键进行 update / delete 操作

set null: 在父表上 update / delete 记录时, 将子表上匹配记录的列设为 null, 要注意子表的外键列不能为 not null

5 SQL Data Types and Schemas

- User-Defined Types

- **create type** construct in SQL creates user-defined type

create type *Dollars* as numeric (12,2) final

- **create table** *department*
(*dept_name* **varchar** (20),
building **varchar** (15),
budget *Dollars*);

- Domains

- **create domain** construct in SQL-92 creates user-defined domain types

create domain *person_name* **char**(20) not null

6 Authorization

SQL 标准包括 **select**、**insert**、**update** 和 **delete** 权限。权限 **所有权限** (all privileges) 可以用作所有允许权限的简写形式。一个创建了新关系的用户将自动被授予该关系上的所有权限。

SQL 数据定义语言包括授予和收回权限的命令。**grant** 语句用来授予权限。此语句的基本形式为： [

```
grant <权限列表>  
on <关系名或视图名>  
to <用户/角色列表>;
```

权限列表使得一个命令可以授予多个权限。角色的概念将在后面 4.6.2 节讨论。

我们使用 **revoke** 语句来收回权限。此语句的形式与 **grant** 几乎是一样的：

```
revoke <权限列表>  
on <关系名或视图名>  
from <用户/角色列表>;
```

因此，要收回前面我们所授予的那些权限，我们书写下列语句：

```
revoke select on department from Amit, Satoshi;  
revoke update (budget) on department from Amit, Satoshi;
```

7 Triggers

ch 6 Database Design Using the E-R Model

1 Outline of the ER Model

- 主要三个要素

entity sets

relationship sets

attributes

1.1 Entity Sets 实体集

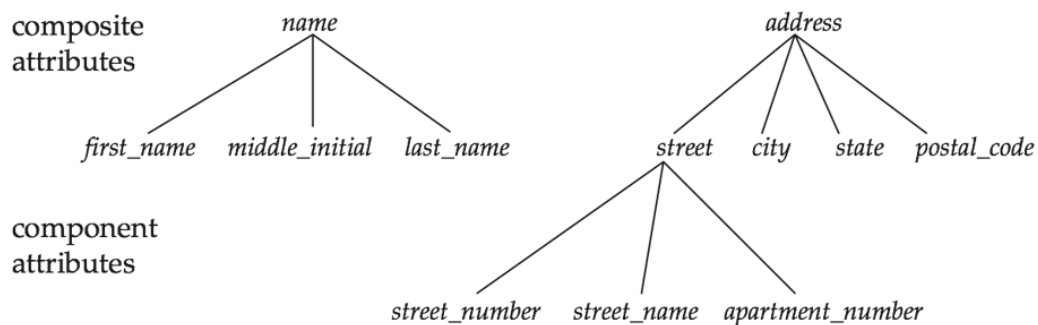
An **entity** is an object that exists and is distinguishable from other objects.

An **entity set** is a set of entities of the same type that share the same properties

1.2 Attributes

- **Simple** and **composite** attributes

Composite Attributes



- **Single-valued** and **multivalued** attributes

Example: multivalued attribute: phone_numbers

- **Derived** attributes

Can be computed from other attributes

1.3 Relationship Sets

- 指多个实体间的相互关联

- **Redundant Attributes 冗余属性**

- Suppose we have entity sets:

- *instructor*, with attributes: *ID*, *name*, *dept_name*, *salary*
 - *department*, with attributes: *dept_name*, *building*, *budget*

- We model the fact that each instructor has an associated department using a relationship set *inst_dept*

- The attribute *dept_name* appears in both entity sets. Since it is the primary key for the entity set *department*, it replicates information present in the relationship and is therefore redundant in the entity set *instructor* and needs to be removed.



- Degree of a Relationship Set

参与联系集的实体集的数目称为联系集的度 (degree)。二元联系集的度为 2；三元联系集的度为 3。

1.4 Mapping Cardinality Constraints 映射基数约束

- 分为 1 对 1，1 对多，多对 1，多对多
- total, partial

如果实体集 *E* 中的每个实体都参与到联系集 *R* 的至少一个联系中，实体集 *E* 在联系集 *R* 中的参与称为**全部 (total)** 的。如果 *E* 中只有部分实体参与到 *R* 的联系中，实体集 *E* 到联系集 *R* 的参与称为**部分 (partial)** 的。在图 7-5a 中，*B* 在联系集中的参与是全部的，而 *A* 在联系集中的参与是部分的；在图 7-5b

1.5 弱实体集 Weak Entity Sets

没有主键的实体集

A weak entity set is one whose existence is dependent on another entity, called its **identifying entity**. In E-R diagrams, a weak entity set is depicted via a **double rectangle**.

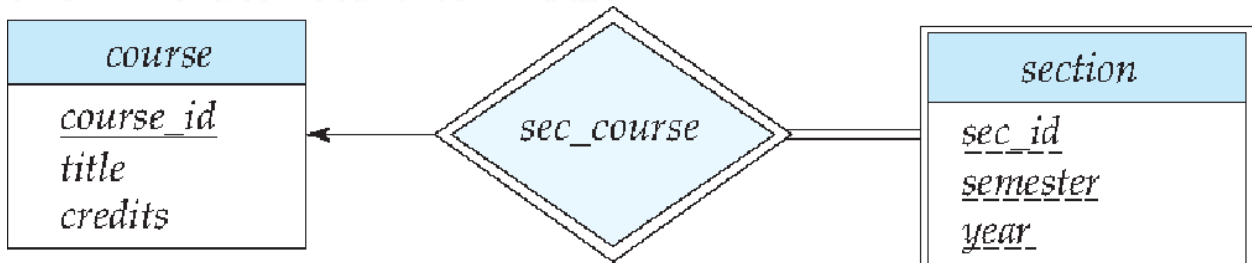
We underline the discriminator of a weak entity set with a **dashed line**.

弱实体集的概念对上述想法进行了正式的规定。没有足够的属性以形成主码的实体集称作**弱实体集 (weak entity set)**。有主码的实体集称作**强实体集 (strong entity set)**。

弱实体集必须与另一个称作**标识 (identifying) 或属主实体集 (owner entity set)** 的实体集关联才能有意义。每个弱实体必须和一个标识实体关联；也就是说，弱实体集**存在依赖 (existence dependent)** 于标识实体集。我们称标识实体集**拥有 (own)** 它所标识的弱实体集。将弱实体集与其标识实体集相联的联系称为**标识性联系 (identifying relationship)**。

虽然弱实体集没有主码，但是我们仍需要区分依赖于特定强实体集的弱实体集中的实体的方法。弱实体集的**分辨符 (discriminator)** 是使得我们进行这种区分的属性集合。例如，弱实体集 *section* 的分辨符由属性 *sec_id*、*year* 以及 *semester* 组成，因为对每门课程来说，这个属性集唯一标识了这门课程的一次开课。弱实体集的分辨符也称为该实体集的**部分码**。

弱实体集的主码由**标识实体集的主码加上该弱实体集的分辨符**构成。在实体集 *section* 的例子中，它的主码是 {*course_id*, *sec_id*, *year*, *semester*}，其中 *course_id* 是标识实体集 *course* 的主码，{*sec_id*, *year*, *semester*} 区分同一门课程的不同 *section* 实体。



2 E-R Diagrams

7.5.1 基本结构

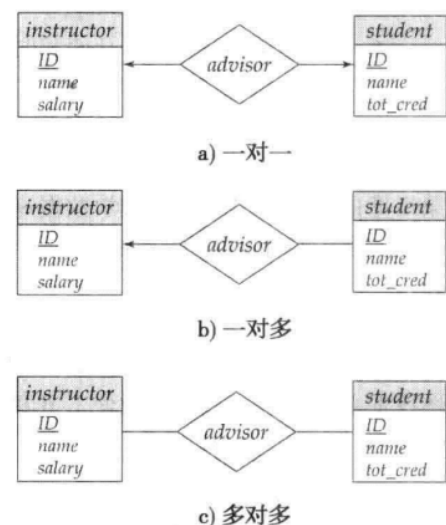
E-R 图包括如下几个主要构件：

- **分成两部分的矩形**代表实体集。本书中有阴影的第一部分包含实体集的名字，第二部分包含实体集中所有属性的名字。
- **菱形**代表联系集。
- **未分割的矩形**代表联系集的属性。构成主码的属性以下划线标明。
- **线段**将实体集连接到联系集。
- **虚线**将联系集属性连接到联系集。
- **双线**显示实体在联系集中的参与度。
- **双菱形**代表连接到弱实体集的标志性联系集（我们将在 7.5.6 节讲述标志性联系集和弱实体集）。

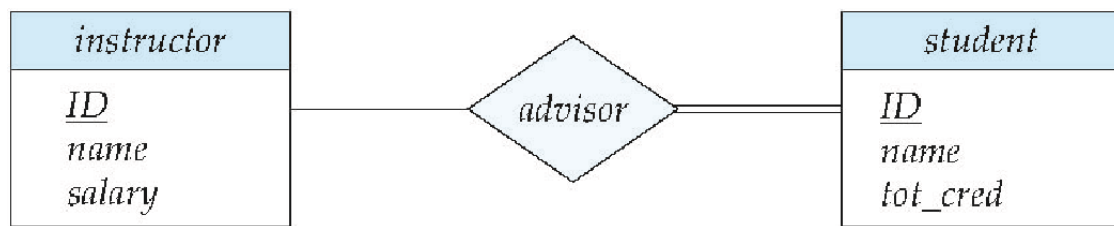
2.1 Cardinality Constraints 映射基数

•

- **一对一**：我们从联系集 *advisor* 向实体集 *instructor* 和 *student* 各画一个箭头（见图 7-9a）。这表示一名教师可以指导至多一名学生，并且一名学生可以有至多一位导师。
- **一对多**：我们从联系集 *advisor* 画一个箭头到实体集 *instructor*，以及一条线段到实体集 *student*（见图 7-9b）。这表示一名教师可以指导多名学生，但一名学生可以有至多一位导师。
- **多对一**：我们从联系集 *advisor* 画一条线段到实体集 *instructor*，以及一个箭头到实体集 *student*。这表示一名教师可以指导至多一名学生，但一名学生可以有多位导师。
- **多对多**：我们从联系集 *advisor* 向实体集 *instructor* 和 *student* 各画一条线段（见图 7-9c）。这表示一名教师可以指导多名学生，并且一名学生可以有多位导师。

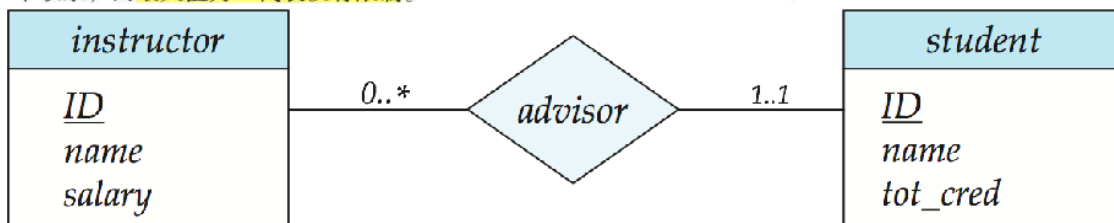


- **Total** and **Partial** Participation

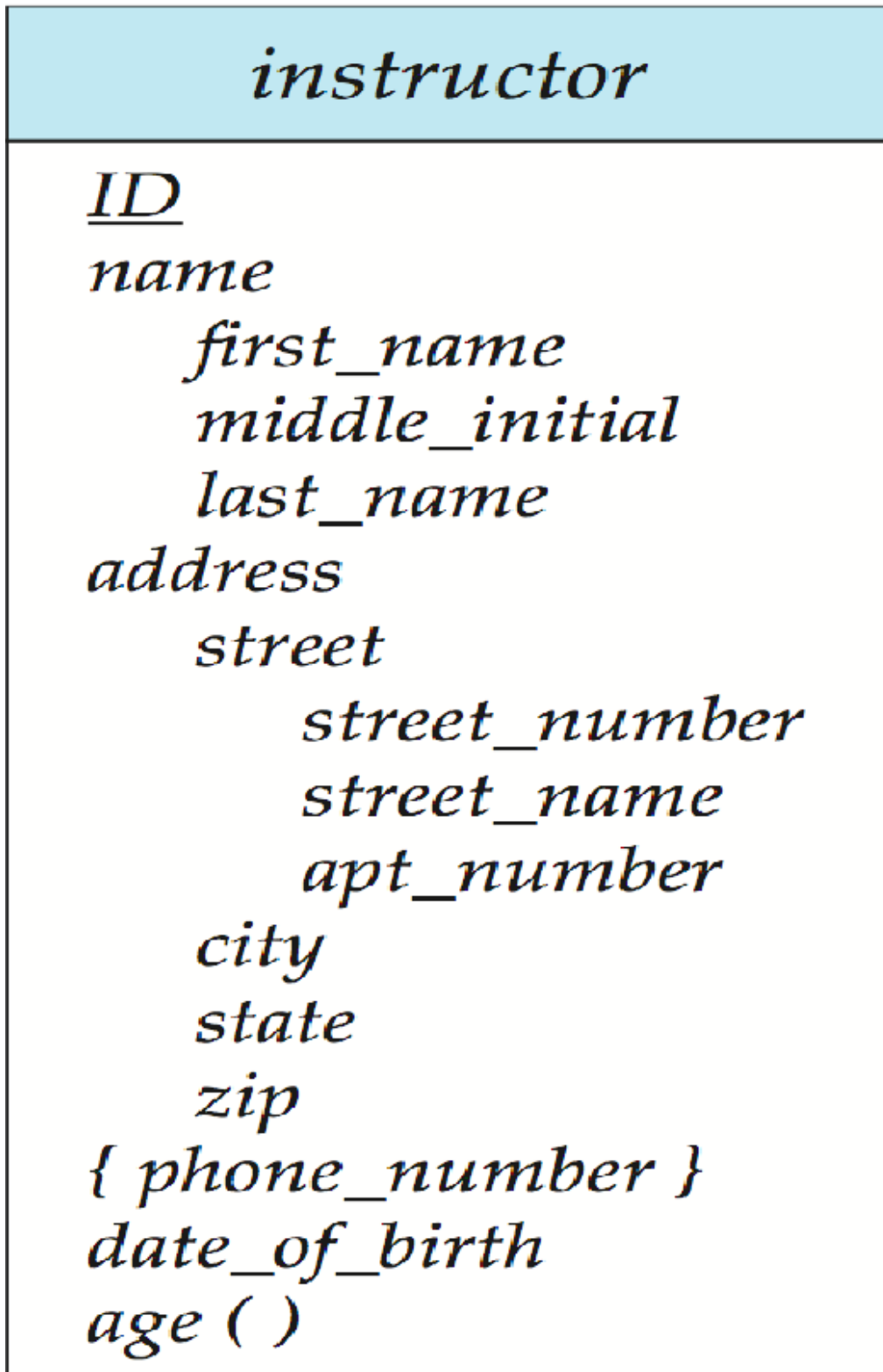


- Notation for Expressing More Complex Constraints

可以有一个关联的最大和最小的映射基数，用 $l..h$ 的形式表示，其中 l 表示最小的映射基数，而 h 表示最大的映射基数。最小值为 1 表示这个实体集在该联系集中全部参与，即实体集中的每个实体在联系集中的至少一个联系中出现。最大值为 1 表示这个实体参与至多一个联系，而最大值为 $*$ 代表没有限制。



- Notation to Express Entity with Complex Attributes



3 Advanced Topics

3.1 Specialization

在实体集内部进行分组的过程

- **Disjoint** (正交/不相交)

an entity can belong to **only one** lower-level entity set

- **Overlapping** (重叠)

an entity can belong to more than one lower-level entity set

- **total**

an entity must belong to one of the lower-level entity sets

- **partial**

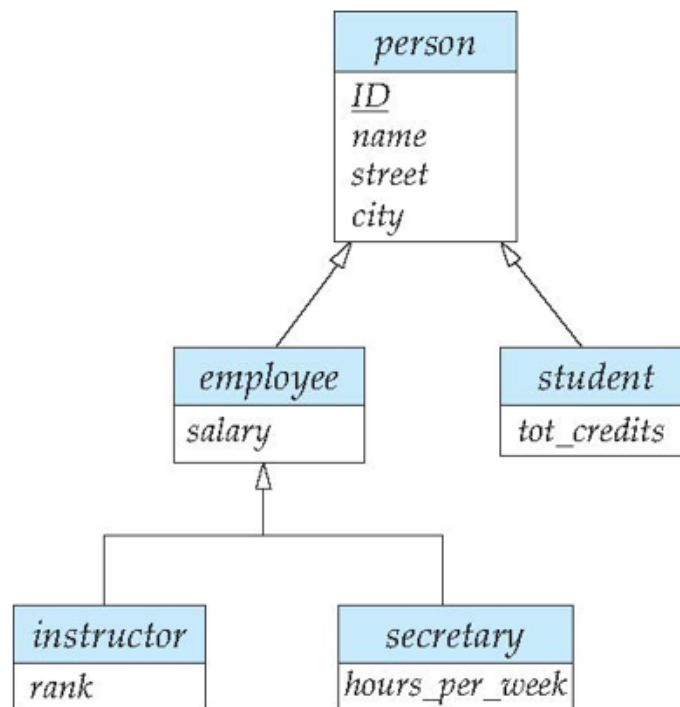
an entity need not belong to one of the lower-level entity sets

我们在 E-R 图中描述特化的方法取决于一个实体集是否可能属于多个特化实体集或者它是否必须属于至多一个特化实体集。前者(允许多个集)称为重叠特化(overlapping specialization)，后者(允许至多一个)称为不相交特化(disjoint specialization)。对于一个重叠特化(例如 *student* 和 *employee* 作为 *person* 的特化的情况)，分开使用两个箭头。对于一个不相交特化(例如 *instructor* 和 *secretary* 作为 *employee* 的特化的情况)，使用一个箭头。特化关系还可能形成超类 - 子类(superclass-subclass)联系。高层和底层实体集按普通实体集表示——即包含实体集名称的矩形。

■ Overlapping – *employee* and *student*

■ Disjoint – *instructor* and *secretary*

■ Total and partial



3.2 Generalization 特化/泛化

把特化反过来

Disjoint , total Specialization/Generalization

Disjoint, partial Specialization/Generalization

Overlapping, partial Specialization/Generalization

Overlapping, total Specialization/Generalization

3.3 Aggregation 聚集

但是，按照这种方法产生的图存在冗余信息，因为在 *eval_for* 中的每个 *instructor*、*student*、*project* 组合肯定也在 *proj_guide* 中。如果 *evaluation* 是一个值而不是一个实体，我们可以将 *evaluation* 作为联系集 *proj_guide* 的一个多值属性。然而，当某个 *evaluation* 和其他实体相关联时，这种方法不可行；例如，每份评估报告可能和负责评估报告的后续处理以发放奖学金的 *secretary* 相关联。

对类似上述情况建模的最好办法是使用聚集。聚集 (aggregation) 是一种抽象，通过这种抽象，联系被视为高层实体。这样，对于我们的例子，我们将联系集 *proj_guide* (关联实体集 *instructor*、*student* 和 *project*) 看成一个名为 *proj_guide* 的高层实体集。这种实体集可以像对任何其他实体集一样来处理。这样我们就可以在 *proj_guide* 和 *evaluation* 之间创建一个二元联系 *eval_for* 来表示某个 *evaluation* 对应哪个 (*student*，*projec*，*instructor*) 组合。图 7-23 所示为用聚集概念来表示以上这种情况。

7.8.6 转换为关系模式

我们现在开始介绍扩展的 E-R 特性如何转换

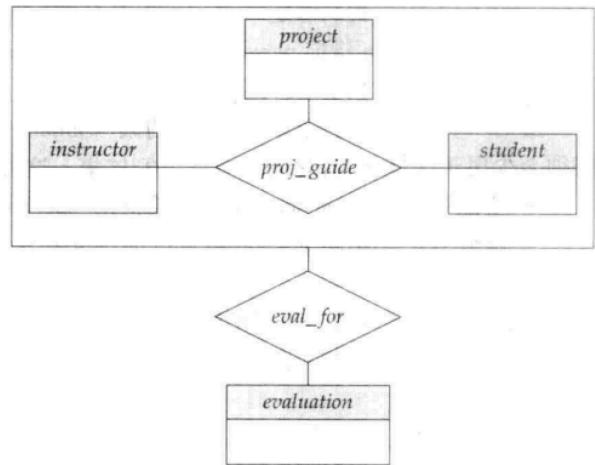


图 7.22 用聚集概念的 E-R 图

4 Design Issues

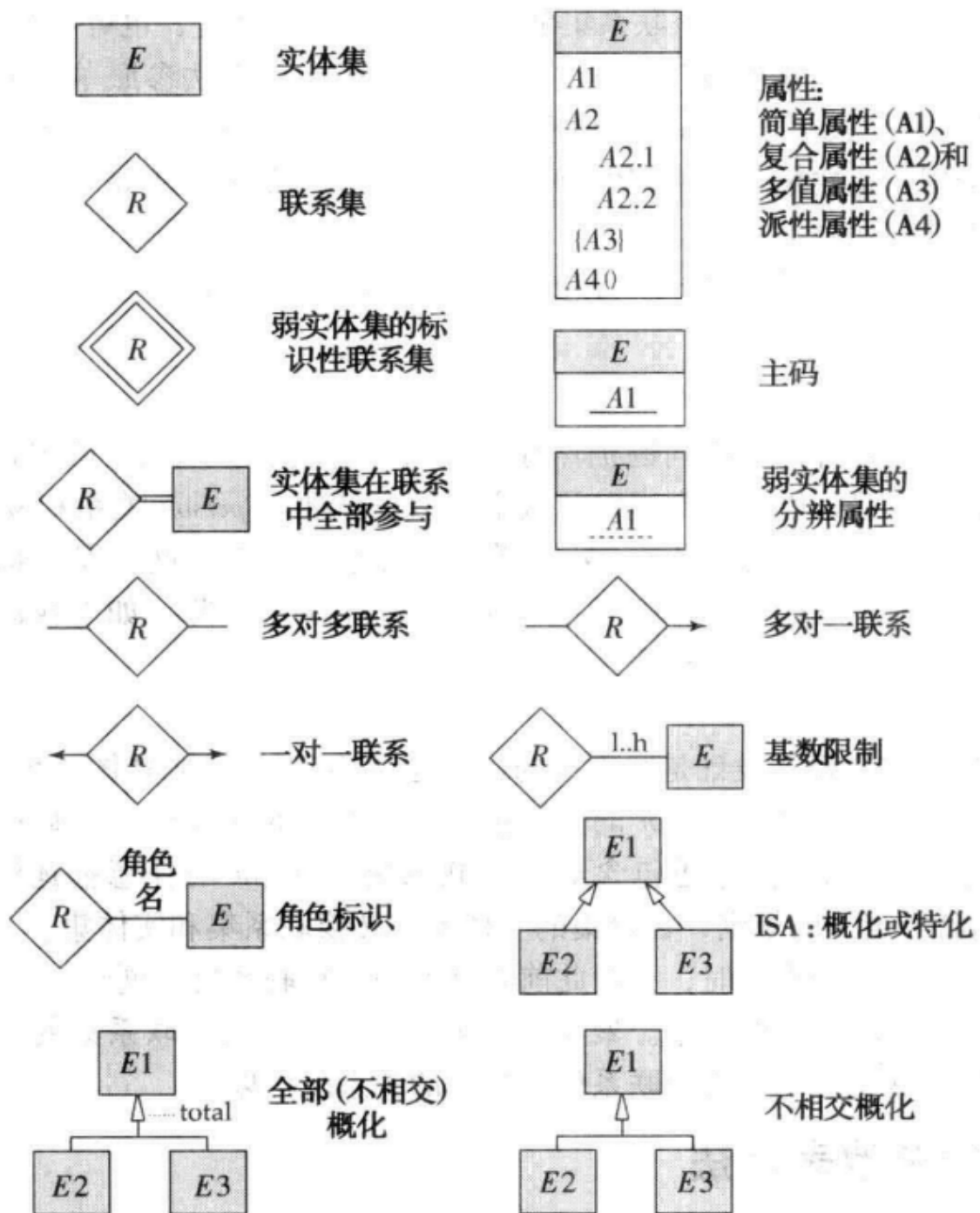
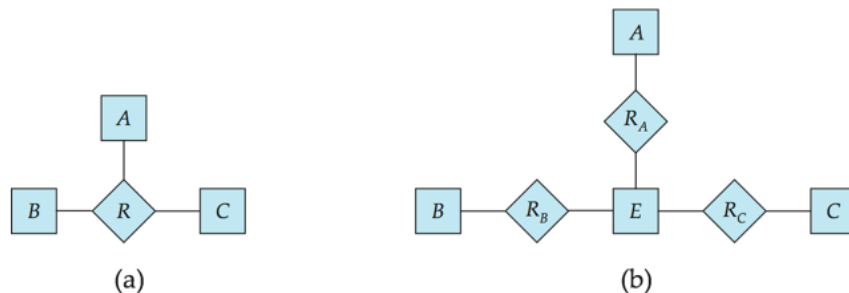


图 7-24 E-R 图表示法中使用的符号

- 将多元联系转换为二元联系

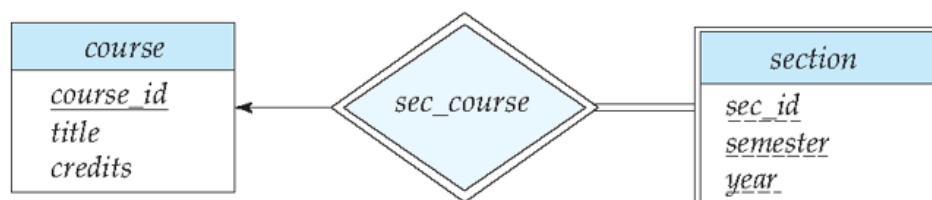
- Replace R between entity sets A , B and C by an entity set E , and three relationship sets:
 1. R_A , relating E and A
 2. R_B , relating E and B
 3. R_C , relating E and C
- Create an identifying attribute for E and add any attributes of R to E
- For each relationship (a_i, b_i, c_i) in R , create
 1. a new entity e_i in the entity set E
 2. add (e_i, a_i) to R_A
 3. add (e_i, b_i) to R_B
 4. add (e_i, c_i) to R_C



5 Reduction to Relation Schemas 转换为关系模式

强实体集直接转，弱实体集需要加入依赖的实体集的主键

- A strong entity set reduces to a schema with the same attributes
 $student(\underline{ID}, name, tot_cred)$
- A weak entity set becomes a table that includes a column for the primary key of the identifying strong entity set
 $section(\underline{course_id}, \underline{sec_id}, sem, year)$



复合属性的每个子属性需要单独写出来

忽略多值属性，多值属性单独写一个模式EM，模式EM具有对应于E的主键的属性，以及一个对应于多值属性M的属性。

• Representation of Entity Sets with Composite Attributes

为每个子属性创建一个单独的属性

```
instructor(ID, first_name, middle_initial, last_name,  
           street_number, street_name, apt_number,  
           city, state, zip_code, date_of_birth)
```

对于一个多值属性 M ，构建关系模式 R ，该模式包含一个对应于 M 的属性 A ，以及对应于 M 所在的实体集或联系集的主码的属性。

例如，考虑图 7-11 中的 E-R 图，它描绘了包含多值属性 $phone_number$ 的实体集 $instructor$ 。 $instructor$ 的主码是 ID 。为这个多值属性构建一个关系模式：

4

```
instructor_phone(ID, phone_number)
```

• Representing Relationship Sets 关系集

- 对于多对多的二元联系，参与实体集的主码属性的并集成为主码。
- 对于一对一的二元联系集，任何一个实体集的主码都可以选作主码。这个选择是任意的。
- 对于多对一或一对多的二元联系集，联系集中“多”的那一方的实体集的主码构成主码。

多对一和一对多的关系集，如果在“多”侧是完全的（total），可以通过在“多”侧添加一个额外的属性来表示，该属性包含“一”侧的主键。

对于一对一的关系集，可以选择任一侧作为“多”侧。

也就是说，可以在对应于这两个实体集的任一表中添加一个额外的属性。如果“多”侧的参与是部分的（partial），那么用“多”侧对应的模式中的一个额外属性来替换一个模式可能会导致空值（null values）。

• Redundancy of Schemas

对应于将弱实体集与其识别强实体集链接起来的关系集的模式是冗余的。

Chapter 7 Relational Database Design

• Functional Dependencies (函数依赖)

约束。考虑一个关系模式 $r(R)$ ，令 $\alpha \subseteq R$ 且 $\beta \subseteq R$ 。

- 给定 $r(R)$ 的一个实例，我们说这个实例满足 (satisfy) 函数依赖 $\alpha \rightarrow \beta$ 的条件是：对实例中所有元组对 t_1 和 t_2 ，若 $t_1[\alpha] = t_2[\alpha]$ ，则 $t_1[\beta] = t_2[\beta]$ 。
- 如果在 $r(R)$ 的每个合法实例中都满足函数依赖 $\alpha \rightarrow \beta$ ，则我们说该函数依赖在模式 $r(R)$ 上成立 (hold)。

使用函数依赖这一概念，我们说如果函数依赖 $K \rightarrow R$ 在 $r(R)$ 上成立，则 K 是 $r(R)$ 的一个超码。也就是说，如果对于 $r(R)$ 的每个合法实例，对于实例中每个元组对 t_1 和 t_2 ，凡是 $t_1[K] = t_2[K]$ ，总有 $t_1[R] = t_2[R]$ (即 $t_1 = t_2$)，则 K 是一个超码。☺

example

函数依赖使我们可以表示不能用超码表示的约束。在 8.1.2 节中我们曾考虑模式：

inst_dept (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)

在该模式中函数依赖 $dept_name \rightarrow budget$ 成立，因为对于每个系（由 *dept_name* 唯一标识）都存在唯一的预算数额。

- Trivial dependency 平凡依赖

有些函数依赖称为平凡的 (trivial)，因为它们在所有关系中都满足。例如， $A \rightarrow A$ 在所有包含属性 *A* 的关系中满足。从字面上看函数依赖的定义，我们知道，对所有满足 $t_1[A] = t_2[A]$ 的元组 t_1 和 t_2 ，有 $t_1[A] = t_2[A]$ 。同样， $AB \rightarrow A$ 也在所有包含属性 *A* 的关系中满足。一般地，如果 $\beta \subseteq \alpha$ ，则形如 $\alpha \rightarrow \beta$ 的函数依赖是平凡的。

- Transitive dependency 传递依赖

- A functional dependency is transitive if:

$\alpha \rightarrow \beta$, $(\beta \not\subseteq \alpha)$, $\beta \not\rightarrow \alpha$, $\beta \rightarrow \gamma$,

Then γ is transitive dependency on α .

- Partial dependency 部分依赖

- A functional dependency is partial if:

$\alpha \rightarrow \beta, \gamma \subset \alpha, \gamma \rightarrow \beta$

β is partially dependent on α .

CTO (CNO, TNO, ADDRESS, OFFICE) :

$CNO \rightarrow TNO, TNO \rightarrow ADDRESS$

partial dependency:

$(CNO, TNO) \rightarrow ADDRESS$

- Logically imply 逻辑蕴含

更正式地，给定关系模式 $r(R)$ ，如果 $r(R)$ 的每一个满足 F 的实例也满足 f ，则 R 上的函数依赖 f 被 r 上的函数依赖集 F 逻辑蕴含 (logically imply)。

- Closure of a Set of Functional Dependencies (函数依赖集闭包)

F^+

- Armstrong's Axioms

➤ **reflexivity** (自反律) : if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$

➤ **Augmentation** (增广律) : if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$

➤ **transitivity** (传递律) : if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

- 合并律 (union rule)。若 $\alpha \rightarrow \beta$ 和 $\alpha \rightarrow \gamma$ 成立, 则 $\alpha \rightarrow \beta\gamma$ 成立。
- 分解律 (decomposition)。若 $\alpha \rightarrow \beta\gamma$ 成立, 则 $\alpha \rightarrow \beta$ 和 $\alpha \rightarrow \gamma$ 成立。
- 伪传递律 (pseudotransitivity rule)。若 $\alpha \rightarrow \beta$ 和 $\gamma\beta \rightarrow \delta$ 成立, 则 $\alpha\gamma \rightarrow \delta$ 成立。

- 计算 F^+ 过程

```
 $F^+ = F$ 
repeat
  for each  $F^+$  中的函数依赖  $f$ 
    在  $f$  上应用自反律和增补律
    将结果加入到  $F^+$  中
  for each  $F^+$  中的一对函数依赖  $f_1$  和  $f_2$ 
    if  $f_1$  和  $f_2$  可以使用传递律结合起来
      将结果加入到  $F^+$  中
until  $F^+$  不再发生变化
```

图 8-7 计算 F^+ 的过程

1 Closure of Attribute Sets 属性集的闭包

如果 $\alpha \rightarrow B$ ，我们称属性 B 被 α 函数确定 (functionally determine)。要判断集合 α 是否为超码，我们必须设计一个算法，用于计算被 α 函数确定的属性集。一种方法是计算 F^+ ，找出所有左半部为 α 的函数依赖，并合并这些函数依赖的右半部。但是这么做开销很大，因为 F^+ 可能很大。

在本节后面我们将看到，一个用于计算被 α 函数确定的属性集的高效算法不仅可以用来判断 α 是否为超码，还可以用于其他的一些任务。

令 α 为一个属性集。我们将函数依赖集 F 下被 α 函数确定的所有属性的集合称为 F 下 α 的闭包，记为 α^+ 。图 8-8 是以伪码写的计算 α^+ 的算法。输入是函数依赖集 F 和属性集 α 。输出存储在变量 $result$ 中。

```

result :=  $\alpha$ ;
repeat
  for each 函数依赖  $\beta \rightarrow r$  in  $F$  do
    begin
      if  $\beta \subseteq result$  then  $result := result \cup r$ ;
    end
until ( $result$  不变)

```

图 8-8 计算 F 下 α 的闭包 α^+ 的算法

- Uses of Attribute Closure

1. Testing for superkey
2. Testing functional dependencies
3. Computing closure of F

- 为了判断 α 是否为超码，我们计算 α^+ ，检查 α^+ 是否包含 R 中的所有属性。
- 通过检查是否 $\beta \subseteq \alpha^+$ ，我们可以检查函数依赖 $\alpha \rightarrow \beta$ 是否成立 (或换句话说，是否属于 F^+)。也就是说，我们用属性闭包计算 α^+ ，看它是否包含 β 。本章后面我们将会看到这种检查非常有用。
- 该算法给了我们另一种计算 F^+ 的方法：对任意的 $\gamma \subseteq R$ ，我们找出闭包 γ^+ ；对任意的 $S \subseteq \gamma^+$ ，我们输出一个函数依赖 $\gamma \rightarrow S$ 。

- Equivalent FD Sets

■ Let F 、 G be two sets of FDs,

if $F^+ = G^+$

We see that F and G are equivalent (等价) .

$$F^+ = G^+ \Leftrightarrow F \subseteq G^+, G \subseteq F^+$$

$$(F^+ \subseteq G^+, G^+ \subseteq F^+)$$

2 Canonical Cover /Minimal Cover 正则覆盖/最小覆盖

- Extraneous Attributes (无关属性)

如果去除函数依赖中的一个属性不改变该函数依赖集的闭包，则称该属性是**无关的 (extraneous)**。
无关属性 (extraneous attribute) 的形式化定义如下：考虑函数依赖集 F 及 F 中的函数依赖 $\alpha \rightarrow \beta$ 。

- 如果 $A \in \alpha$ 并且 F 逻辑蕴含 $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ ，则属性 A 在 α 中是无关的。
- 如果 $A \in \beta$ 并且函数依赖集 $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ 逻辑蕴含 F ，则属性 A 在 β 中是无关的。

例如，假定我们在 F 中有函数依赖 $AB \rightarrow C$ 和 $A \rightarrow C$ ，那么 B 在 $AB \rightarrow C$ 中是无关的。再比如，假定我们在 F 中有函数依赖 $AB \rightarrow CD$ 和 $A \rightarrow C$ ，那么 C 在 $AB \rightarrow CD$ 的右半部中是无关的。

• Testing Extraneous Attributes

下面介绍如何有效**检验一个属性是否无关**。令 R 为一关系模式，且 F 是在 R 上成立的给定函数依赖集。考虑依赖 $\alpha \rightarrow \beta$ 中的一个属性 A 。

- 如果 $A \in \beta$ ，为了检验 A 是否是无关的，考虑集合

$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$$
 并检验 $\alpha \rightarrow A$ 是否能够由 F' 推出。为此，计算 F' 下的 α^+ (α 的闭包)；如果 α^+ 包含 A ，则 A 在 β 中是无关的。
- 如果 $A \in \alpha$ ，为了检验 A 是否是无关的，令 $\gamma = \alpha - \{A\}$ ，并且检查 $\gamma \rightarrow \beta$ 是否可以由 F 推出。为此，计算在 F 下的 γ^+ (γ 的闭包)；如果 γ^+ 包含 β 中的所有属性，则 A 在 α 中是无关的。

• Canonical Cover/Minimal Cover

F 的**正则覆盖 (canonical cover)** F_c 是一个依赖集，使得 F 逻辑蕴含 F_c 中的所有依赖，并且 F_c 逻辑蕴涵 F 中的所有依赖。此外， F_c 必须具有如下性质：

- F_c 中任何函数依赖都不含无关属性。
- F_c 中函数依赖的左半部都是唯一的。即， F_c 中不存在两个依赖 $\alpha_1 \rightarrow \beta_1$ 和 $\alpha_2 \rightarrow \beta_2$ ，满足 $\alpha_1 = \alpha_2$ 。

函数依赖集 F 的正则覆盖可以按图 8-9 中描述的那样计算。需要重视的一点是，当检验一个属性是否无关时，检验时用的是 F_c 当前值中的函数依赖，而不是 F 中的依赖。如果一个函数依赖的右半部只包含一个属性，例如 $A \rightarrow C$ ，并且这个属性是无关的，那么我们将得到一个右半部为空的函数依赖。这样的函数依赖应该删除。

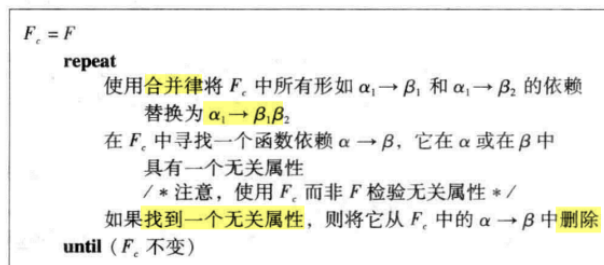


图 8-9 计算正则覆盖

3 Normal Forms

3.1 First Normal Form 1 NF

在关系模型中，我们将属性不具有任何子结构这个思想形式化。**一个域是原子的 (atomic)**，如果该域的元素被认为是不可分的单元。我们称一个关系模式 R 属于**第一范式 (First Normal Form, 1NF)**，如果 R 的所有属性的域都是原子的。

名字的集合是一个非原子值的例子。例如，如果关系 *employee* 的模式包含一个属性 *children*，它的域元素是名字的集合，该模式就不属于第一范式。

3.2 Second Normal Form 2 NF

若 R 是 1 NF，且每个非键属性完全依赖于候选键，则称 R 为 2 NF（消除非键属性对候选键的部分依赖）

A relation schema R is in second normal form (2 NF) if **each attribute A in R** meets **one** of the following criteria:

- It appears in a candidate key;
- It is not partially dependent on a candidate key.

3.3 Third Normal Form 3 NF

若 R 是 2 NF，且非键属性不传递依赖于 R 的候选键，则称 R 是第三范式。

具有函数依赖集 F 的关系模式 R 属于 **第三范式 (third normal form)** 的条件是：对于 F^+ 中所有形如 $\alpha \rightarrow \beta$ 的函数依赖 (其中 $\alpha \subseteq R$ 且 $\beta \subseteq R$)，以下至少一项成立：

- $\alpha \rightarrow \beta$ 是一个平凡的函数依赖。
- α 是 R 的一个超码。
- $\beta - \alpha$ 中的每个属性 A 都包含于 R 的一个候选码中。

3.4 Boyce-Codd Normal Form BCNF

我们能达到的较满意的范式之一是 **Boyce-Codd 范式 (Boyce-Codd Normal Form, BCNF)**。它消除所有基于函数依赖能够发现的冗余，虽然，如我们将在 8.6 节中看到的，可能有其他类型的冗余还保留着。具有函数依赖集 F 的关系模式 R 属于 BCNF 的条件是，对 F^+ 中所有形如 $\alpha \rightarrow \beta$ 的函数依赖 (其中 $\alpha \subseteq R$ 且 $\beta \subseteq R$)，下面至少有一项成立：

- $\alpha \rightarrow \beta$ 是平凡的函数依赖 (即， $\beta \subseteq \alpha$)。
- α 是模式 R 的一个超码。

如果关系模式 R 是 1 NF，且每个属性都不部分依赖于候选键也不传递依赖于候选键，那么称 R 是 BC 范式。

If a relation is in BCNF it is in 3 NF

3.5 多属性依赖集候选关键字求法：

(1) 将 R 的所有属性分为四类：

L 类：仅出现在 F 的函数依赖左部的属性；

R 类：仅出现在 F 的函数依赖右部的属性；

N 类：在 F 的函数依赖左右两边均未出现的属性；

LR 类：在 F 的函数依赖左右两边均出现的属性；

并令 X 代表 L、N 类， Y 代表 LR 类；

(2) 求 X^+ ，若包含了 R 的所有属性，则 X 即为 R 的唯一候选关键字，转 (5)，否则转 (3)；

(3) 在 Y 中取一属性 A ，求 $(XA)^+$ ，若它包含了 R 的所有属性，则转 (4)，否则，调换一属性反复进行这一过程，直到试完所有 Y 中的属性；

(4) 如果已找出所有的候选关键字，则转 (5)，否则在 Y 中依此取两个、三个，...，求他

们的属性闭包，直到其闭包包含 R 的所有的属性。

(5) 停止，输出结果。

3.6 范式级别判断

范式级别判断

题目：U=ABCDE, F={A→C, BC→D, CD→A, AB→E}

第一步（求候选码）：求候选码的过程略去，结果为{(AB), (BC)}

第二步（判断1NF）：呃，肯定都是1NF，不是的话这题不用出了

第三步（写出非主属性、候选码）：这一步是为了方便后续判断。

非主属性是未出现在候选码中的属性，本题为DE。候选码为AB和BC。

如下列出：

非主属性：DE；候选码：AB、BC

在题干中标记非主属性（粉色字体）：

F={A→C, BC→D, CD→A, AB→E}

第四步（判断2NF）：2NF的定义为非主属性应完全函数依赖于任一候选码。即看非主属性的左侧的推断条件中是否出现了候选码，也就是题目中D和E的左侧是否出现了AB或BC（AB或BC必须一起打包出现，不能只出现单个的A，B之类）。举例来说，假如题目中存在A→D，D是非主属性，而左侧只有A，没有和B一起出现，即D不是完全函数依赖于候选码AB的，它只部分依赖于AB中的A。观察当前题目，非主属性D和E的左侧均是候选码之一，并不存在这种情况，那么它是满足2NF的。

第五步（判断3NF）：在满足2NF的前提下（注意满足2NF之后再判断3NF哦~），3NF的定义为不存在非主属性对任一候选码的传递函数依赖。举个例子，如果有AB→D，又有D→E，即非主属性E通过非主属性D作为桥梁才能推出，这就是传递依赖。尽管候选码AB能推出D和E，但E是间接推出的，也不可以出现，这咋。继续观察题目，并没有这种情况，那么是满足3NF的。

第六步（判断BCNF）：在满足3NF的前提下，BCNF的条件是每一个决定因素（即每一个依赖关系的左侧）都包含码。即决定因素必须包含候选码中的任意一个。即在本题中，每个决定因素要么有AB，要么有BC。要么A和B成对出现，要么B和C成对出现。该题中，第一个关系A→C就只出现了A，A没有和B一起出现，所以本题不满足BCNF。注意这里和2NF的区别，2NF只看针对非主属性的左侧，而BCNF要看所有依赖关系的左侧。

因此本题为3NF。（玄学：实在不会就写3NF，3NF出现的概率大些）

4 Decomposition

- Lossless-join decomposition 无损分解

分解时没有信息损失

我们可以用函数依赖来说明什么情况下分解是无损的。令 R 、 R_1 、 R_2 和 F 如上。 R_1 和 R_2 是 R 的无损分解，如果以下函数依赖中至少有一个属于 F^+ ：

- $R_1 \cap R_2 \rightarrow R_1$
- $R_1 \cap R_2 \rightarrow R_2$

- Dependency preservation 保持依赖

dependency preserving:

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

$$F_i = \{X \rightarrow Y \mid X \rightarrow Y \in F^+ \cap XY \subseteq R_i\}$$

- Testing for Dependency Preservation

方法的思想。该验证方法对 F 中的每一个 $\alpha \rightarrow \beta$ 使用下面的过程：

```
result =  $\alpha$ 
repeat
  for each 分解后的  $R_i$ 
     $t = (result \cap R_i)^+ \cap R_i$ 
     $result = result \cup t$ 
until ( $result$  没有变化)
```

图 8-10 保持依赖性的验证

这里的属性闭包是函数依赖集 F 下的。如果 $result$ 包含 β 中的所有属性，则函数依赖 $\alpha \rightarrow \beta$ 保持。分解是保持依赖的当且仅当上述过程中 F 的所有依赖都保持。

5 3 NF Decomposition Algorithm

```
令  $F_c$  为  $F$  的正则覆盖；
 $i := 0$ ；
for each  $F_c$  中的函数依赖  $\alpha \rightarrow \beta$ 
   $i := i + 1$ ；
   $R_i := \alpha\beta$ ；
if 模式  $R_j$ ,  $j = 1, 2, \dots, i$  都不包含  $R$  的候选码
then
   $i := i + 1$ ；
   $R_i := R$  的任意候选码；
```


6 BCNF Decomposition Algorithm

8.5.1.2 BCNF 分解算法

我们现在能给出一个关系模式分解的一般方法以满足 BCNF。图 8-11 所示为实现该过程的一个算法。若 R 不属于 BCNF，则可用这个算法将 R 分解成一组 BCNF 模式 $R_1, R_2, \dots, R_n$ 。这个算法利用违反 BCNF 的依赖进行分解。

```
result := { R };
done := false;
计算  $F^+$ ;
while (not done) do
  if (result 中存在模式  $R_i$  不属于 BCNF)
    then begin
      令  $\alpha \rightarrow \beta$  为一个在  $R_i$  上成立的非平凡函数依赖，满足  $\alpha \rightarrow \beta$  不属于  $F^+$ ，并且  $\alpha \cap \beta = \emptyset$ ;
      result := (result -  $R_i$ )  $\cup$  ( $R_i - \beta$ )  $\cup$  ( $\alpha, \beta$ );
    end
  else done := true;
```

图 8-11 BCNF 分解算法

Chapter 13 Data Storage Structures

1 File Organization

- Fixed-Length Records

- Simple approach:

- Store record i starting from byte $n * (i - 1)$, where n is the size of each record.
 - Record access is simple but records may cross blocks

Modification: do not allow records to cross block boundaries

- Deletion of record i : alternatives:

- move records $i + 1, \dots, n$ to $i, \dots, n - 1$
 - move record n to i
 - do not move records, but link all free records on a *free list*

- Variable-Length Records

- Attributes are stored in order
- Variable length attributes represented by fixed size (offset, length), with actual data stored after all fixed length attributes
- Null values represented by null-value bitmap

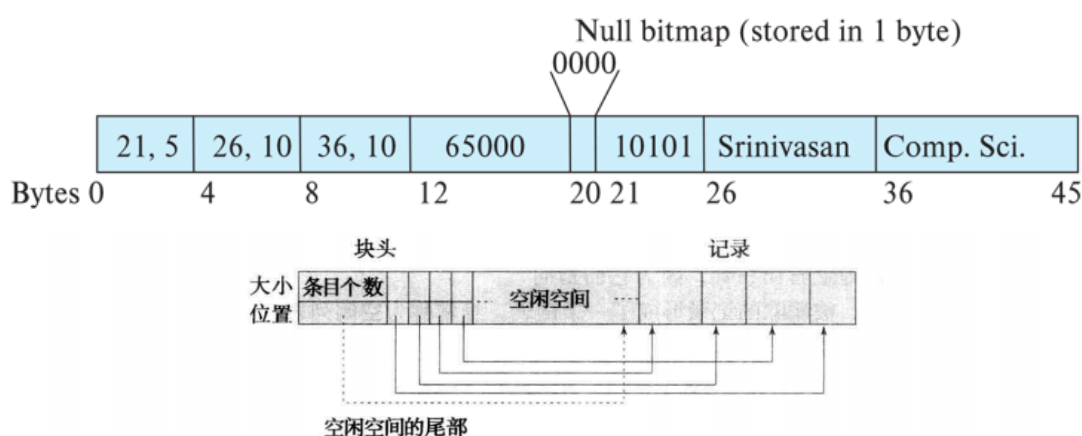


图 10-9 分槽的页结构

我们接下来处理在块中存储变长记录的问题，**分槽的页结构**(slotted-page structure)一般用于在块中组织记录，如图 10-9 所示。每个块的开始处有一个块头，其中包含以下信息：

1. 块头中记录条目的个数。
2. 块中空闲空间的末尾处。
3. 一个由包含记录位置和大小记录组成的数组。

实际记录从块的尾部开始连续排列。块中空闲空间是连续的，在块头数组的最后一个条目和第一条记录之间。如果插入一条记录，在空闲空间的尾部给这条记录分配空间，并且将包含这条记录大小和位置的条目添加到块头中。

2 Organization of Records in Files

- **堆文件组织**(heap file organization)。一条记录可以放在文件中的任何地方，只要那个地方有空间存放这条记录。记录是没有顺序的。通常每个关系使用一个单独的文件。
- **顺序文件组织**(sequential file organization)。记录根据其“搜索码”的值顺序存储。10.6.1 节描述了这种组织方式。
- **散列文件组织**(hashing file organization)。在每条记录的某些属性上计算一个散列函数。散列函数的结果确定了记录应放到文件的哪个块中。第 11 章描述这种组织方式。它与那一章所描述的索引结构密切相关。

Records of each relation may be stored in a separate file. In a **clustering file organization** (聚集) records of several different relations can be stored in the same file

3 Data –Dictionary Storage

Data dictionary (also called system catalog) stores **metadata**: that is, data about data

- Information about relations
 - names of relations
 - names and types of attributes of each relation
 - names and definitions of views
 - integrity constraints
- User and accounting information, including passwords
- Statistical and descriptive data
 - number of tuples in each relation
- Column-Oriented Storage

Store each attribute of a relation separately

Chapter 14 Indexing

1 Basic Concepts

- **Search Key** – An attribute or set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**索引项) of the form

search-key	pointer
------------	---------

- 两种基本的索引类型:
 - **Ordered indices**: search keys are stored in sorted order
 - **Hash indices**: search keys are distributed uniformly across “buckets” using a “hash function”.
- 评价因素:
 - Access types
 - Access time
 - Insertion time
 - Deletion time
 - Space overhead

2 Ordered Indices: Primary index, Secondary index

- **Primary index (主索引)** : in a **sequentially ordered 有序文件** file, the index whose search key specifies the sequential order of the file.
 - Also called **clustering index 聚集索引**
 - The search key of a primary index is **usually** but not necessarily the primary key. 不一定建立在主键上，只要顺序一致即可
- **Index-sequential file**: ordered sequential file with a primary index.
- **Secondary index 辅助索引**: an index whose search key specifies an order **different from** the sequential order of the file. Also called **non-clustering index**.

- **Dense Indices 稠密索引**

每一个 search key 值都有一个索引项 index entries

- **Sparse Indices 稀疏索引**

只为 search key 某些值建立索引项

指针。为了定位一条记录，我们找到其最大搜索码值小于或等于所查找记录的搜索码值的索引项。然后从该索引项指向的记录开始，沿着文件中的指针查找，直到找到所需记录为止。

- **Multilevel Index 多级索引**

outer index – a sparse index of primary index

inner index – the primary index file

- **Index Update: Deletion**
 - **Single-level index deletion:**

- **Dense indices –**

- Delete the pointers(stores pointers to all records);

- Update the pointer (stores pointers to the first record);

- **Sparse indices – Nothing done;**

- replace with next search-key value;

- updates the pointer;

- **Index Update: Insertion**

Perform a lookup using the search-key value appearing in the record to be inserted.

Dense indices –insert index/add pointer/update pointer

Sparse indices – no change / insert index / update pointer

- **Primary and Secondary Indices**

Secondary indices have to be dense

When a file is modified, every index on the file must be updated, Updating indices imposes overhead on database modification.

3 B+-Tree Index

All paths from root to leaf are of the same length

知道了叶结点的结构之后，我们来看一看搜索码值是如何赋给特定结点的。每个叶结点最多可有 $n-1$ 个值。我们允许叶结点包含的值的个数最少为 $\lceil (n-1)/2 \rceil$ 。在 B⁺ 树例子中 $n=4$ ，每个叶子必须包含最少两个并且最多 3 个值。

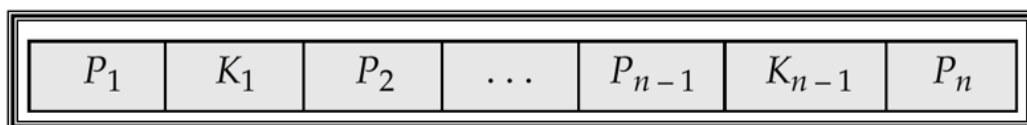
B⁺ 树的非叶结点 (nonleaf node) 形成叶结点上的一个多级 (稀疏) 索引。非叶结点的结构和叶结点的相同，只不过非叶结点中所有的指针都是指向树中结点的指针。一个非叶结点可以容纳最多 n 个指针，同时必须至少容纳 $\lceil n/2 \rceil$ 个指针。结点的指针数称为该结点的扇出。非叶结点也称为内部结点 (internal node)。

- Special cases:

If the root is not a leaf, it has at least 2 children.

If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.

Typical node:



➤ K_i are the search-key values

➤ P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).

The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \dots < K_{n-1}$$

- **Queries on B+-Trees**

function *find*(*v*)

1. $C = \text{root}$
2. **while** (C is not a leaf node)
 1. Let i be least number s.t. $V \leq K_i$.
 2. **if** there is no such number i *then*
 3. Set $C = \text{last non-null pointer in } C$
 4. **else if** ($v = C.K_i$) Set $C = P_{i+1}$
 5. **else set** $C = C.P_i$
3. **if** for some i , $K_i = V$ **then** return $C.P_i$
4. **else** return null /* no record with search-key value v exists. */

4 Hash Indices

到目前为止，我们一直假设，当插入一条记录时，记录映射到的桶有存储记录的空间。如果桶没有足够的空间，就会发生**桶溢出**(bucket overflow)。桶溢出的发生可能有以下几个原因：

- **桶不足。**桶数目(用 n_b 表示)的选择必须使 $n_b > n_r/f_r$, 其中 n_r 表示将要存储的记录总数, f_r 表示一个桶中能存放的记录数目。当然, 这种表示是以在选择散列函数时记录总数已知为前提的。
- **偏斜。**某些桶分配到的记录比其他桶多, 所以即使其他桶仍有空间, 某个桶也仍可能溢出。这种情况称为桶偏斜(skew)。偏斜发生的原因有两个:
 1. 多条记录可能具有相同的搜索码。
 2. 所选的散列函数可能会造成搜索码的分布不均。

5 Index Definition in SQL

- Create an index

```
create [unique] index <index-name> on <relation-
name>
    <attribute-list>)
```

E.g.: **create index** *b-index* **on** *branch(branch-name)*

- create cluster index
 - create noncluster index
 - create unique index

- To drop an index

drop index <index-name>

Chapter 15 Query Processing

1 Basic Steps in Query Processing

查询解析 Parsing and translation

查询优化 Optimization

查询执行 Evaluation

Chapter 16 Query Optimization

1 Heuristic optimization (启发式优化)

1.1 Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections, 选择串接律:

$$\delta_{C_1 \text{ and } C_2 \dots C_n} (E) = \delta_{C_1} (\delta_{C_2} \dots (\delta_{C_n} (E)) \dots)$$

2. Selection operations are commutative, 选择交换律

$$\sigma_{\theta_1} (\sigma_{\theta_2} (E)) = \sigma_{\theta_2} (\sigma_{\theta_1} (E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted, 投影串接律

$$\pi_{L_1} (\pi_{L_2} \dots (\pi_{L_n} (E)) \dots) = \pi_{L_1} (E)$$

$L_1 \subseteq L_K$ K 为 $1 \dots n$, 即 L_1 为其最小的投影属性集合。

4. Selections can be combined with Cartesian products and theta joins.

$$a. \sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$$

$$b. \sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$$

6.(a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_2 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3 .

7. The selection operation distributes over the theta join operation under the following two conditions:

(a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

(b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2 .

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

$$\delta_{\text{sex}='f'}(S \bowtie SC) = (\delta_{\text{sex}='f'}(S)) \bowtie (SC)$$

$$\delta_{\text{sex}='f' \cap \text{cname}='db'}(S \bowtie SC) = (\delta_{\text{sex}='f'}(S)) \bowtie (\delta_{\text{cname}='db'}(SC))$$

8. The projections operation distributes over the theta join operation as follows:

(a) If θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = (\Pi_{L_1}(E_1)) \bowtie_{\theta} (\Pi_{L_2}(E_2))$$

(b) Consider a join $E_1 \bowtie_{\theta} E_2$.

★ Let L_1 and L_2 be sets of attributes from E_1 and E_2 , respectively .

★ Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and

★ let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_{\theta} E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_{\theta} (\Pi_{L_2 \cup L_4}(E_2)))$$

9. The set operations union and intersection are commutative

$$\begin{aligned}E_1 \cup E_2 &= E_2 \cup E_1 \\E_1 \cap E_2 &= E_2 \cap E_1\end{aligned}$$

10. Set union and intersection are associative.

$$\begin{aligned}(E_1 \cup E_2) \cup E_3 &= E_1 \cup (E_2 \cup E_3) \\(E_1 \cap E_2) \cap E_3 &= E_1 \cap (E_2 \cap E_3)\end{aligned}$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2)$$

and similarly for $\cup$ and $\cap$ in place of $-$

Also: $\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - E_2$
and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$



- 1 选择、投影尽早执行，减少中间结果；
- 2 笛卡尔乘积和选择组合成连接操作；
- 3 相同关系的多个选择和投影操作可同时执行，减少扫描；
- 4 投影和相邻的操作结合，减少为投影而进行单独的 I/O 操作；
- 5 存储公共的常用的结果

- 步骤：

- (1) 构造查询树；
- (2) 按优化规则、变化规则进行优化。

- 查询树

- (1) 表示关系代数表达式的语法树形结构；
叶节点：关系；
非叶节点：各种关系代数操作；
从底向上执行。

(2) 从 SQL 构造查询树：

- FROM 对应笛卡尔乘积；
WHERE 对应选择操作；
SELECT 对应投影操作。

- 优化算法

输入：一个关系代数表达式的语法树；

输出：计算表达式的一个优化程序。

执行步骤：对查询树从顶至下优化：

- (1) 利用**选择串接律**，将**选择操作**分成单个独立；
 - (2) 对于每个**选择操作**，利用选择的交换律、和投影的交换律、和连接/笛卡尔乘积的分配律、和集合操作的分配律，使其尽可能移到靠近叶节点；
 - (3) 对于每个**投影操作**，利用投影串接律、和选择的交换律、和连接/笛卡尔乘积的分配律、和集合操作的分配律，使其尽可能移到靠近叶节点或删除某些投影操作；
 - (4) 使用选择、投影的串接律，选择、投影的交换律，将串接的多个选择和投影组合成一个；
 - (5) 利用笛卡尔乘积/连接/结合操作的结合律，按照小关系先执行的原则，安排操作顺序；
 - (6) 组合笛卡尔乘积和相继的选择操作为连接操作；
 - (7) 对每个叶节点加必要的投影操作，消除对查询无用的属性；
 - (8) 分组。每个二元运算（笛卡尔乘积、连接、集合操作）与其直接祖先（不越过别的二元运算节点）的一元运算节点（投影、选择），分为一组，如其子孙节点一直到叶节点皆为选择或投影，也并入该组。使每组的操作可由单个存取程序完成。
-

Chapter 17 Transactions 事务

1 ACID Properties

Atomicity（原子性）

Consistency（一致性）

Isolation（隔离性）

Durability（保持性）

Atomicity（原子性）： Either all operations of the transaction are properly reflected in the database or none are. **Transaction-management component**

Consistency（一致性）： Execution of a transaction in isolation preserves the consistency of the database.

Isolation（隔离性）： each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.

Concurrency-control schemes

Durability（保持性）： After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

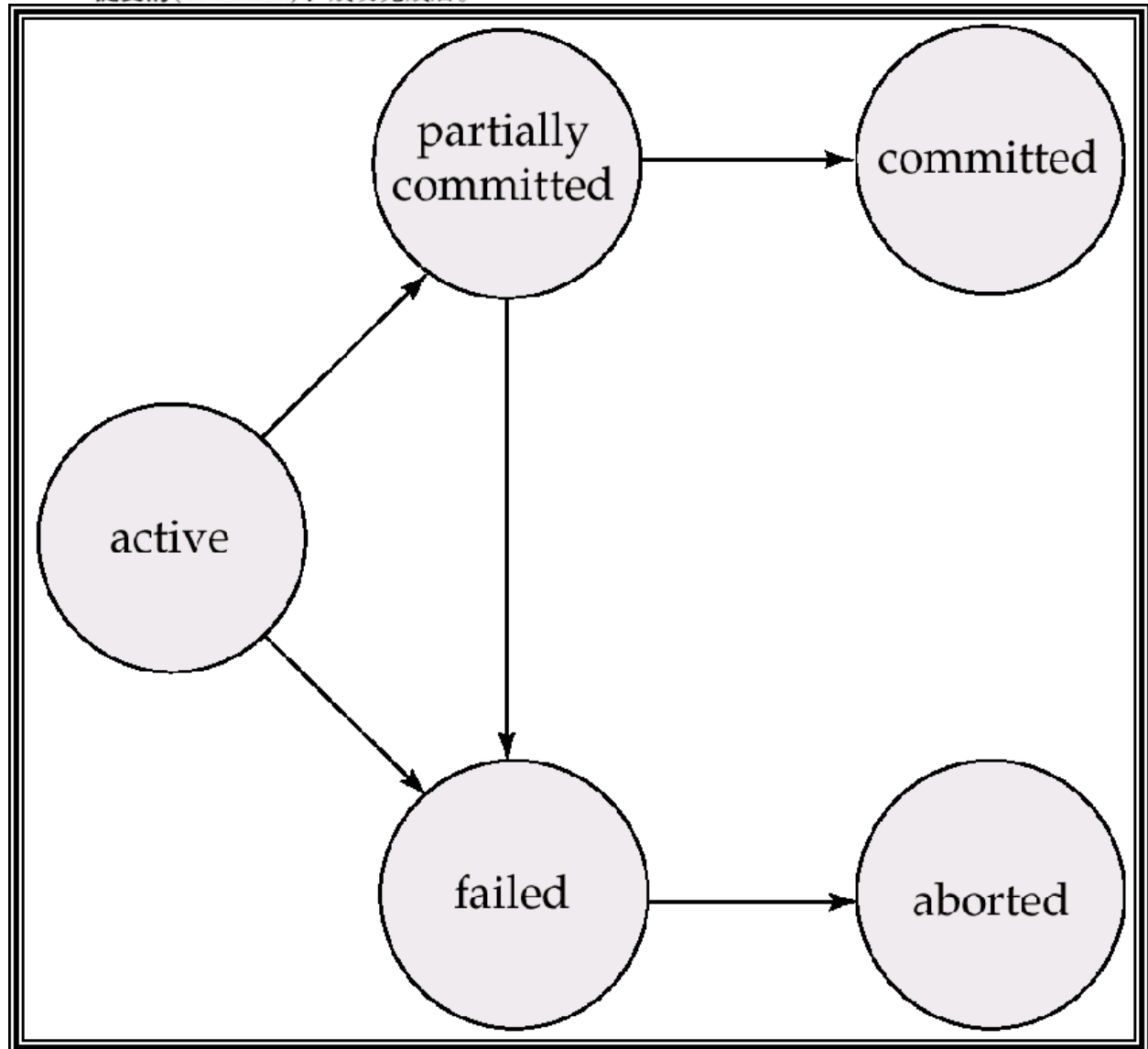
recovery-management component



2 Transaction State

我们需要更准确地定义一个事务成功完成的含义。为此我们建立了一个简单的抽象事务模型。事务必须处于以下状态之一。

- 活动的(active)：初始状态，事务执行时处于这个状态。
- 部分提交的(partially committed)：最后一条语句执行后。
- 失败的(failed)：发现正常的执行不能继续后。
- 中止的(aborted)：事务回滚并且数据库已恢复到事务开始执行前的状态后。
- 提交的(committed)：成功完成后。



3 Implementation of Atomicity and Durability

4 Concurrent Executions

5 Serializability

- Conflict Serializability

Conflict Operations:

1. $l_i = \text{read}(Q)$, $l_j = \text{write}(Q)$. They conflict.
2. $l_i = \text{write}(Q)$, $l_j = \text{read}(Q)$. They conflict
3. $l_i = \text{write}(Q)$, $l_j = \text{write}(Q)$. They conflict
4. $l_i = \text{read}(Q)$, $l_j = \text{read}(Q)$. l_i and l_j don't conflict
5. Operations on different data items, don't conflict

▪ If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.

▪ We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule.

判断一个调度是否冲突可串行化: **Precedence graph** 前驱图、优先图:

(precedence graph)。该图由两部分组成 $G = (V, E)$, 其中 V 是顶点集, E 是边集, 顶点集由所有参与调度的事务组成, 边集由满足下列三个条件之一的边 $T_i \rightarrow T_j$ 组成:

- 在 T_j 执行 $\text{read}(Q)$ 之前, T_i 执行 $\text{write}(Q)$ 。
- 在 T_j 执行 $\text{write}(Q)$ 之前, T_i 执行 $\text{read}(Q)$ 。
- 在 T_j 执行 $\text{write}(Q)$ 之前, T_i 执行 $\text{write}(Q)$ 。

如果优先图中存在边 $T_i \rightarrow T_j$, 则在任何等价于 S 的串行调度 S' 中, T_i 必出现在 T_j 之前。

T_3	T_4
read(Q)	write(Q)
write(Q)	

图 14-9 调度 7

6 Recoverability

调度 9 是不可恢复调度的一个例子。一个**可恢复调度 (recoverable schedule)** 应满足: 对于每对事务 T_i 和 T_j , 如果 T_j 读取了之前由 T_i 所写的数据项, 则 T_i 先于 T_j 提交。例如, 如果要使调度 9 是可恢复的, 则 T_7 应该推迟到 T_6 提交后再提交。

14.7.2 无级联调度

即使一个调度是可恢复的, 要从事务 T_i 的故障中正确恢复, 可能需要回滚若干事务。当其他事务读取了由事务 T_i 所写数据而时就会发生这种情形。

T_6	T_7
read(A)	read(A) commit
write(A)	
read(B)	

图 14-14 调度 9:

7 cascadeless

说, **无级联调度 (cascadeless schedule)** 应满足: 对于每对事务 T_i 和 T_j , 如果 T_j 读取了先前由 T_i 所写的数据项, 则 T_i 必须在 T_j 这一读操作前提交。容易验证每一个无级联调度也都是可恢复的调度。

8 Implementation of Isolation

- Levels of Isolation in SQL-92

Serializable（可串行化） — 隔离最高级别

Repeatable read(可重复读) — only committed records to be read. 在开始读取数据（事务开启）时，不再允许修改操作

- Repeated reads of same record must return same value.
- However, a transaction may not be serializable – it may find some records inserted by a transaction but not find others.

Read committed（已提交读） — only committed records can be read. 一个事务要等另一个事务提交后才能读取数据。

- Successive reads of record may return different (but committed) values.

Read uncommitted（未提交读） — even uncommitted records may be read.

脏读：读到了别的事务回滚前的脏数据。

不可重复读：当前事务先进行了一次数据读取，然后再次读取到的数据是别的事务修改成功的数据，导致两次读取到的数据不匹配。

幻读：事务 A 首先根据条件查询得到 N 条数据，事务 B 改变了这 N 条数据之外的 M 条或者增添了 M 条符合事务 A 查询条件的数据，导致事务 A 再次搜索发现有 N+M 条数据了，就产生了幻读。

当前事务读第一次取到的数据比后来读取到数据条目不一致。

四种隔离级别可能导致的问题： *DataBase System Concepts*

Serializable (可串行化)： 最严格的级别，事务串行执行，资源消耗最大；

REPEATABLE READ(重复读)： 避免了“脏读取”和“不可重复读取”的情况，但不能避免“幻读”。

READ COMMITTED (提交读)： 避免了“脏读”，但不能避免“幻读”和“不可重复读取”。

Read Uncommitted(未提交读)： 会导致“脏读”、“幻读”和“不可重复读取”。

Chapter 18 Concurrency Control

- Lock-Based Protocols 基于锁的协议

exclusive (X) mode

shared (S) mode

- The Two-Phase Locking Protocol

保证可串行性的一个协议是两阶段封锁协议(two-phase locking protocol)。该协议要求每个事务分两个阶段提出加锁和解锁申请。

1. 增长阶段(growing phase): 事务可以获得锁, 但不能释放锁。
2. 缩减阶段(shrinking phase): 事务可以释放锁, 但不能获得新锁。

级联回滚可以通过将两阶段封锁修改为严格两阶段封锁协议(strict two-phase locking protocol)加以避免。这个协议除了要求封锁是两阶段之外, 还要求事务持有的所有排他锁必须在事务提交后方可释放。这个要求保证未提交事务所写的任何数据在该事务提交之前均以排他方式加锁, 防止其他事务读这些数据。

另一个两阶段封锁的变体是强两阶段封锁协议(rigorous two-phase locking protocol), 它要求事务提交之前不得释放任何锁。我们很容易验证在强两阶段封锁条件下, 事务可以按其提交的顺序串行化。

T_5	T_6	T_7
lock-X(A)		

Lock Conversions

以上观察提示我们对基本的两阶段封锁协议加以修改, 使之允许锁转换(lock conversion)。我们将提供一种将共享锁升级为排他锁, 以及将排他锁降级为共享锁的机制。我们用升级(upgrade)表示从共享到排他的转换, 用降级(downgrade)表示从排他到共享的转换。锁转换不能随意进行。锁升级只能发生在增长阶段, 而锁降级只能发生在缩减阶段。

- Multiple Granularity

树中每个结点都可以单独加锁。正如我们在两阶段封锁协议中所做的那样, 我们将使用共享(shared)锁与排他(exclusive)锁。当事务对一个结点加锁, 或为共享锁或为排他锁, 该事务也以同样类型的锁隐式地封锁这个结点的全部后代结点。例如, 若事务 T_i 给图 15-15 中的 F_i 显式(explicit lock)地加排他锁, 则事务 T_i 也给所有属于该文件的记录隐式(implicit lock)地加排他锁。没有必要显式地给 F_i 中的单条记录逐个加锁。

Intention Lock Modes

与共享锁相关联的是一种意向锁, 与排他锁相关联的是另一种意向锁。如果一个结点加上了共享型意向(intention-shared (IS) mode)锁, 那么将在树的较低层进行显式封锁, 但只能加共享锁。类似地, 如果一个结点加排他型意向(intention-exclusive (IX) mode)锁, 那么将在树的较低层进行显式封锁, 可以加排他锁或共享锁。最后, 若一个结点加上了共享排他型意向锁(shared and intention-exclusive (SIX) mode), 则以该结点为根的子树显式地加了共享锁, 并且将在树的更低层显式地加排他锁。这些锁类加锁从根节点向下加, 释放锁从下向上释放

IS: 子为 shared, 父为 IS/S/IX/SIX

IX: 子为 exclusive, 父为 IX/IS

SIX: 子为 X, 父为 S, 则父应该为 SIX

- Deadlock Prevention Strategies

wait-die scheme: 让老事务等待新事务

▪ **wait-die** scheme

If $TS(T_i) < TS(T_j)$ then T_i Waits
Else
RollBack T_i
Restart T_i with the same $TS(T_i)$
Endif

wound-wait scheme: 回滚的是正在执行的事务

▪ **wound-wait** scheme

If $TS(T_i) > TS(T_j)$ then T_i Waits
Else
RollBack T_j
Restart T_j with the same $TS(T_j)$
Endif

- Deadlock Detection
- Deadlock Recovery

Chapter 19 Recovery System

检查点的执行过程如下:

1. 将当前位于主存的所有日志记录输出到稳定存储器。
2. 将所有修改的缓冲块输出到磁盘。
3. 将一个日志记录 **<checkpoint L >** 输出到稳定存储器, 其中 L 是执行检查点时正活跃的事务的列表。

崩溃发生后当数据库系统重启时，恢复动作分两阶段进行：

1. **在重做阶段**，系统通过从最后一个检查点开始正向地扫描日志来重放所有事务的更新。重放的日志记录包括在系统崩溃之前已回滚的事务的日志记录，以及在系统崩溃发生时还没有提交的事务的日志记录。这个阶段还确定在系统崩溃发生时未完成，因此必须回滚事务。这样的未完成事务或者在检查点时是活跃的，因此会出现在检查点记录的事务列表中，或者是在检查点之后开始的；而且，这样的未完成事务在日志中既没有 $\langle T_i, \text{abort} \rangle$ 记录，也没有 $\langle T_i, \text{commit} \rangle$ 记录。

扫描日志的过程中所采取的具体步骤如下：

- a. 将要回滚的事务的列表 **undo-list** 初始设定为 $\langle \text{checkpoint } L \rangle$ 日志记录中的 L 列表。
- b. 一旦遇到形为 $\langle T_i, X_j, V_1, V_2 \rangle$ 的正常日志记录或形为 $\langle T_i, X_j, V_2 \rangle$ 的 redo-only 日志记录，就重做这个操作；也就是说，将值 V_2 写给数据项 X_j 。
- c. 一旦发现形为 $\langle T_i, \text{start} \rangle$ 的日志记录，就把 T_i 加到 **undo-list** 中。
- d. 一旦发现形为 $\langle T_i, \text{abort} \rangle$ 或 $\langle T_i, \text{commit} \rangle$ 的日志记录，就把 T_i 从 **undo-list** 中去掉。

在 redo 阶段的末尾，**undo-list** 包括在系统崩溃之前尚未完成的所有事务，即，既没有提交也没有完成回滚的那些事务。

2. **在撤销阶段**，系统回滚 **undo-list** 中的所有事务。它通过从尾端开始反向扫描日志来执行回滚。
 - a. 一旦发现属于 **undo-list** 中的事务的日志记录，就执行 undo 操作，就像在一个失败事务的回滚过程中发现了该日志记录一样。
 - b. 当系统发现 **undo-list** 中事务 T_i 的 $\langle T_i, \text{start} \rangle$ 日志记录，它就往日志中写一个 $\langle T_i, \text{abort} \rangle$ 日志记录，并且把 T_i 从 **undo-list** 中去掉。
 - c. 一旦 **undo-list** 变为空表，即系统已经找到了开始时位于 **undo-list** 中的所有事务的 $\langle T_i, \text{start} \rangle$ 日志记录，则撤销阶段结束。

当恢复过程的撤销阶段结束之后，就可以重新开始正常的事务处理了。